

ОСНОВЫ Node.js

Знакомство со встроенными модулями и
фреймворком Express



План курса





Что будет на уроке сегодня

-  Модули в Node.js
-  Обзор встроенных модулей
-  Глобальные объекты и функции
-  Работа с модулями
-  Обзор фреймворка express
-  Создание http сервера с помощью express

Модули в Node.js



Определение

Модуль - это файл, код которого можно переиспользовать
в других файлах





Виды модулей

- Встроенные модули - эти модули являются частью Node.js
- Модули NPM - эти модули можно установить с помощью NPM
- Пользовательские модули - это модули, которые вы создаете сами



Экспорт данных из модуля

Из любого модуля можно экспортировать любой набор данных с помощью `module.exports`

```
1 function func() {  
2   console.log('Hello!');  
3 }  
4  
5 const string = "some string";  
6  
7 const number = 10;  
8  
9 const array = [1, 2, 3];  
10  
11 const object = { key: 'value' };  
12  
13 module.exports = { func, string, number, array, object };
```



Модуль должен экспортировать данные
немедленно!





Как нельзя экспортировать

Так модуль ничего не экспортирует!

```
1 setTimeout(() => {  
2     const someVar = 10;  
3     module.exports = { someVar };  
4 }, 0);
```

Нельзя использовать `module.exports` в асинхронных функциях



Как не рекомендуется экспортировать

Такой скрипт ничего не экспортирует

```
1 function exportInFunction() {  
2     const someVar = 10;  
3     module.exports = { someVar };  
4 }
```



Как не рекомендуется экспортировать

Но такой скрипт будет экспортировать данные

```
1 function exportInFunction() {  
2   const someVar = 10;  
3   module.exports = { someVar };  
4 }  
5  
6 exportInFunction();
```

Но делать так не рекомендуется



Лучше всего экспортировать данные в конце скрипта





Модуль не обязательно должен экспортировать данные.
Модуль также может существовать без экспорта





Импорт модулей

Модули любого вида можно импортировать с помощью глобальной функции `require()`

```
1 // Импорт встроенного модуля
2 const http = require('http');
3
4 // Импорт NPM модуля
5 const uuid = require('uuid');
6
7 // Импорт пользовательского модуля
8 const myModule = require('./myModule');
```




Алгоритм импорта

1. Функция `require()` сначала ищет встроенные модули
2. Затем установленные модули с помощью NPM
3. И наконец ищет среди пользовательских модулей

Это упрощенная схема. Подробный алгоритм
расписан в [официальной документации](#)

Пользовательские модули



Пользовательские модули

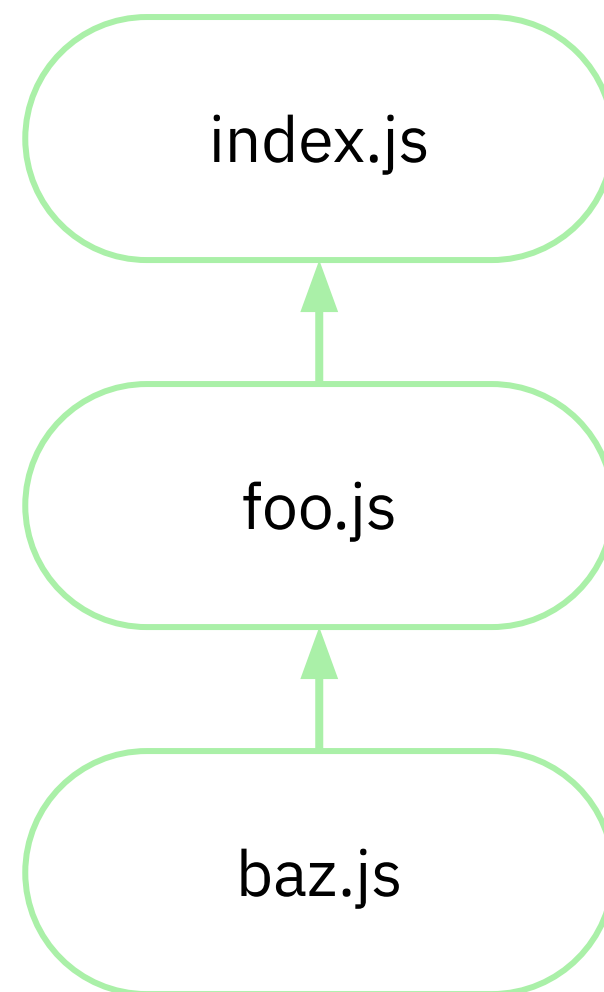
Вы можете импортировать как отдельный файл, так и директорию.

- Для импорта файла, необходимо указать путь к файлу
- Для импорта директории нужно указать путь к директории. При этом в директории должен обязательно находиться файл с именем `index.js`



Импорт по цепочке

Импортируемые модули также могут импортировать другие модули



```
1 // index.js
2 const foo = require('./foo');
3
4 console.log(foo);
```

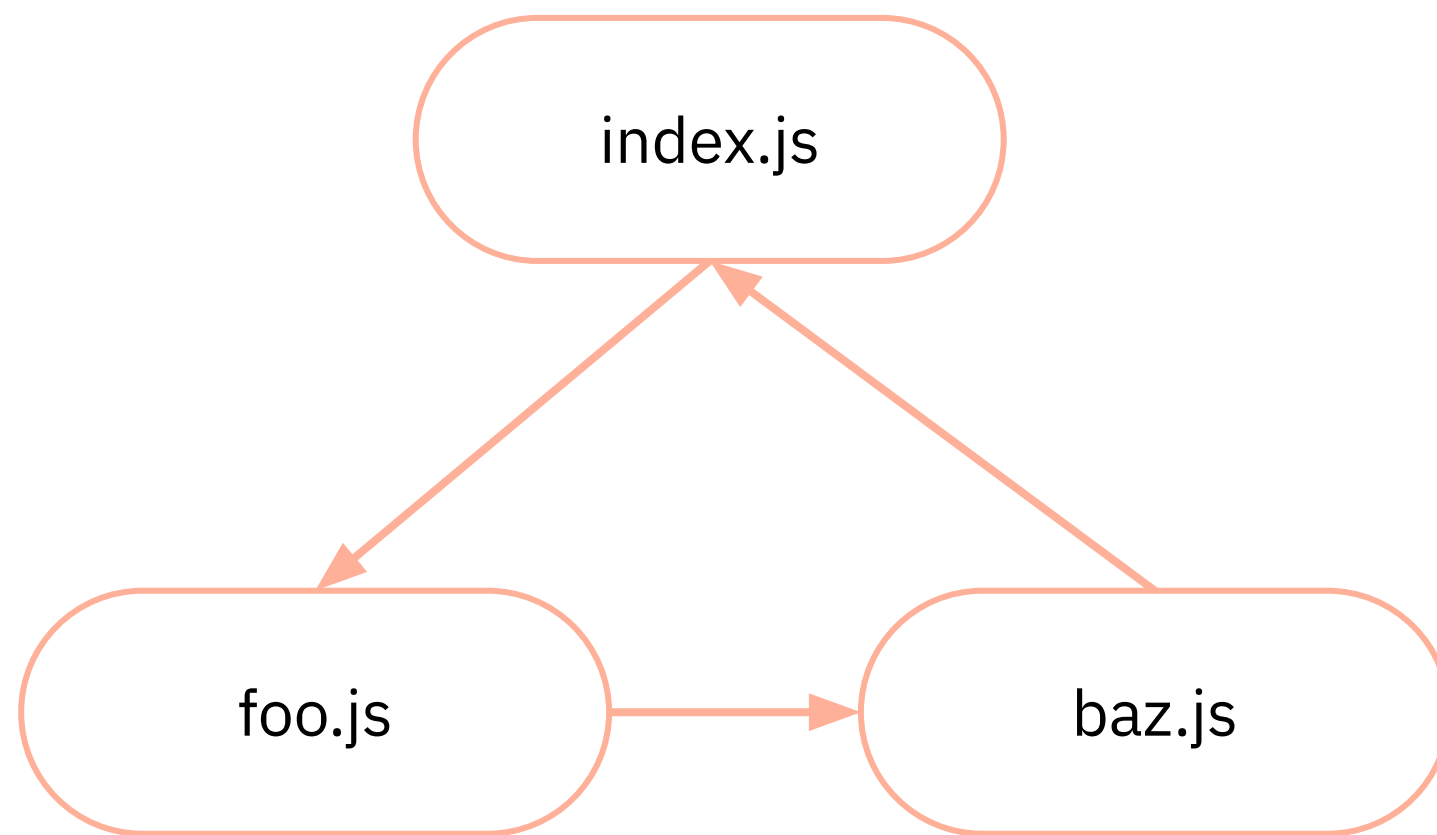
```
1 // foo.js
2 const baz = require('./baz');
3
4 console.log(baz);
5
6 module.exports = 'Hello form foo!';
```

```
1 // baz.js
2 module.exports = 'Hello from baz!';
```

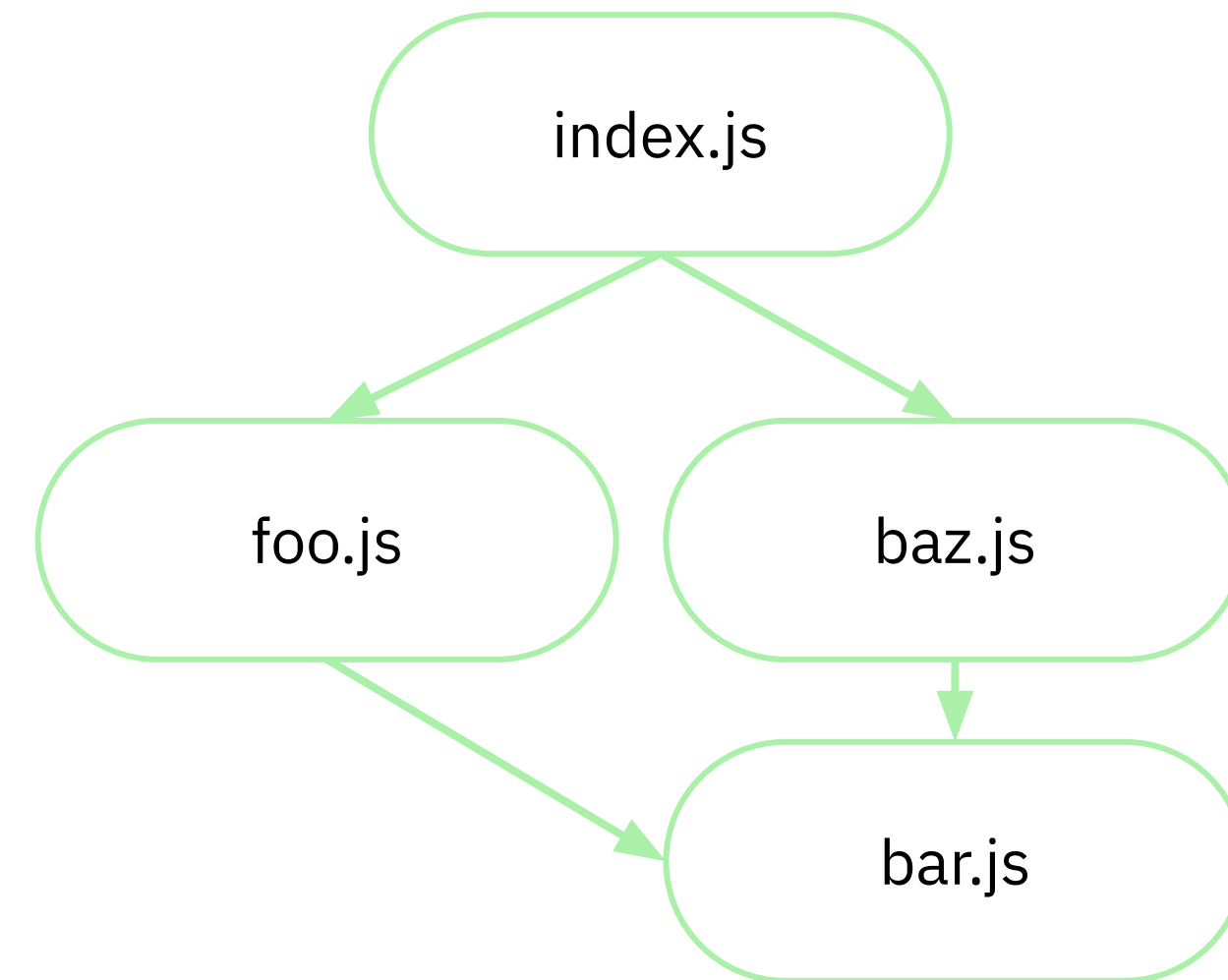


Циклические зависимости

Циклическая зависимость возникает, когда цепочка зависимостей начинается и заканчивается на одном и том же файле



Пример неправильных зависимостей



Пример правильных зависимостей

Глобальные объекты



Объект global

Данный объект содержит в себе набор свойств и функций, которые доступны глобально

```
1 Math.floor(15.5);  
2 // Тоже самое, что и  
3 global.Math.floor(15.5);  
4  
5 global.setTimeout(() => console.log('Hello!'), 1000);  
6 // Тоже самое, что и  
7 setTimeout(() => console.log('Hello!'), 1000);
```

Данный объект записывается в глобальный контекст при старте скрипта



Объект process

Объект process содержит в себе набор свойств и функций для работы с ОС и окружением

```
1 console.log(process.version); // Версия Node.js
2 console.log(process.arch); // Архитектура процессора
3 console.log(process.pid); // Идентификатор текущего процесса в ОС
4 console.log(process.cwd()); // Директория откуда запущен скрипт
```



Свойства `__dirname` и `__filename`

`__dirname` хранит в себе путь к директории в которой хранится текущий скрипт.

`__filename` хранит в себе путь к текущему скрипту

```
1 // Предположим, что скрипт находится тут:  
  //Users/Student/Desktop/index.js  
2  
3 // Выведет в консоль путь к файлу  
4 // /Users/Student/Desktop/index.js  
5 console.log(__filename);  
6  
7 // Выведет в консоль путь к директории хранения файла  
8 // /Users/Student/Desktop  
9 console.log(__dirname);
```

Обзор встроенных модулей



Модуль fs

Модуль fs позволяет работать с файловой системой: читать и записывать файлы

```
1 const fs = require('fs');  
2  
3 fs.readFile('path/to/file', 'utf-8', (err, data) => {  
4     console.log(data);  
5 });
```



Чтение файла

Для чтения файла у модуля fs определена функция readFile

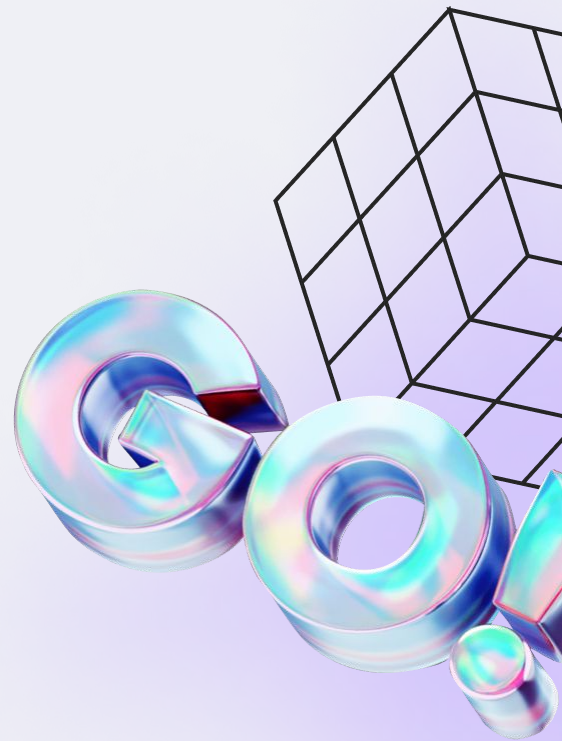
```
1 const fs = require('fs');
2
3 fs.readFile('path/to/file', 'utf-8', (err, data) => {
4     if (err) {
5         console.error(err);
6     } else {
7         console.log(data);
8     }
9 });
```




Вопрос

Что выведет этот скрипт в консоль?

```
1 const fs = require('fs');  
2  
3 fs.readFile(__filename, 'utf-8', (err, data) => {  
4     if (err) {  
5         console.error(err);  
6     } else {  
7         console.log(data);  
8     }  
9 });
```





Ответ

В консоли мы увидим содержимое файла
запущенного скрипта





Запись в файл

Для записи в файл у модуля fs определена функция writeFile

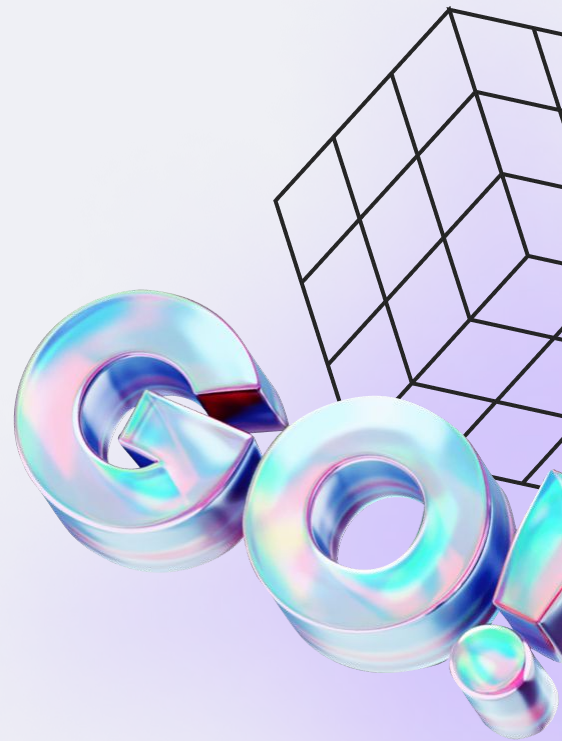
```
1 const fs = require('fs');
2
3 fs.writeFile(__filename, 'console.log("Hello!")', (err) => {
4     if (err) {
5         console.error(err);
6     }
7     console.log('The file was saved!');
8 });
```



Вопрос

Что произойдет, если запустить
этот скрипт?

```
1 const fs = require('fs');  
2  
3 fs.writeFile(__filename, 'console.log("Hello!")', (err) => {  
4     if (err) {  
5         console.error(err);  
6     }  
7     console.log('The file was saved!');  
8 });
```





Ответ

Файл запущенного скрипта будет
перезаписан





Добавление в файл

Чтобы не перезаписывать, а добавлять данные в файл, существует функция `appendFile`

```
1 const fs = require('fs');
2
3 fs.appendFile(__filename, '\nconsole.log("Hello!");', (err) => {
4     if (err) {
5         console.error(err);
6     }
7
8     console.log('The file was saved!');
9 });
```

Данный скрипт добавит вызов `console.log` в конец этого же скрипта



Синхронные и асинхронные методы

Синхронные методы - это аналоги асинхронных. Они делают те же самые действия, но при этом блокируют поток выполнения скрипта

```
1 const fs = require('fs');  
2  
3 const fileData = fs.readFileSync(__filename, 'utf-8');  
4  
5 console.log(fileData);  
6  
7 fs.appendFileSync(__filename, '\nconsole.log("Hello!")');
```

Асинхронные методы возвращают результат выполнения и колбек определять не нужно



Модуль path

Этот модуль помогает работать с путями в файловой системе

```
1 const path = require('path');
2
3 // Выведет в консоль путь /User/Student/Desktop/index.js
4 console.log(path.join('/User/Student', 'Desktop/index.js'));
5 // Выведет в консоль объект пути
6 console.log(path.parse('/User/Student/Desktop/index.js'));
7 // Выведет в консоль путь к директории, убрав название файла
8 console.log(path.dirname('/User/Student/Desktop/index.js'));
9 // Выведет в консоль расширение файла .js
10 console.log(path.extname('/User/Student/Desktop/index.js'));
```



Модуль os

Этот модуль позволяет получить информацию об операционной системе

```
1 const os = require('os');
2
3 // Выведет в консоль информацию о ядрах процессора
4 console.log(os.cpus());
5 // Выведет в консоль информацию об архитектуре процессора
6 console.log(os.arch());
7 // Выведет в консоль количество свободной памяти в байтах
8 console.log(os.freemem());
9 // Выведет в консоль общее количество оперативной памяти в MB
10 console.log(os.totalmem() / 1024 / 1024 / 1024);
```



Какие еще существуют модули?

url - позволяет работать с URL адресами и их компонентами

crypto - позволяет работать с криптографией. Содержит в себе различные криптографические функции

stream - позволяет управлять большими потоками данных

А также многие другие модули.

О всех существующих модулях можно почитать в [официальной документации](#)



Промежуточные итоги

- Мы с вами выяснили, что такое модули и как работает импорт и экспорт
- Поняли, что существует 3 вида модулей: встроенные, внешние(NPM) и пользовательские
- И попробовали поработать со встроенными и с собственными модулями
- Также мы познакомились с глобальными объектами в Node.js
- Теперь пора знакомиться с express =)

Введение в express



Определение

Express - это быстрый, гибкий и минималистичный фреймворк для создания веб-приложений на Node.js





Пример кода на express

Минимальная реализация сервера на express

```
1 const express = require('express');
2
3 const app = express();
4
5 app.get('/', (req, res) => {
6     res.send('<h1>Добро пожаловать на мой сайт!<h1>');
7 });
8
9 const port = 3000;
10
11 app.listen(port, () => {
12     console.log(`Сервер запущен на порту ${port}`);
13 });
```



Установка express

Чтобы начать работать с express, необходимо выполнить три простых шага:

1. Для начала нужно создать новую директорию для проекта
2. Затем в этой директории необходимо инициализировать NPM проект с помощью команды **npm init**
3. И наконец можно устанавливать express с помощью команды **npm install express**



Объекты `req` и `res`

В аргументах колбека обработчика можно получить доступ ко объектам **`req`** и **`res`**

`req` - это объект, который содержит в себе информацию о запросе

`res` - это объект, который содержит методы для работы с ответом на запрос



Несколько обработчиков

Вы можете определить сколько угодно обработчиков

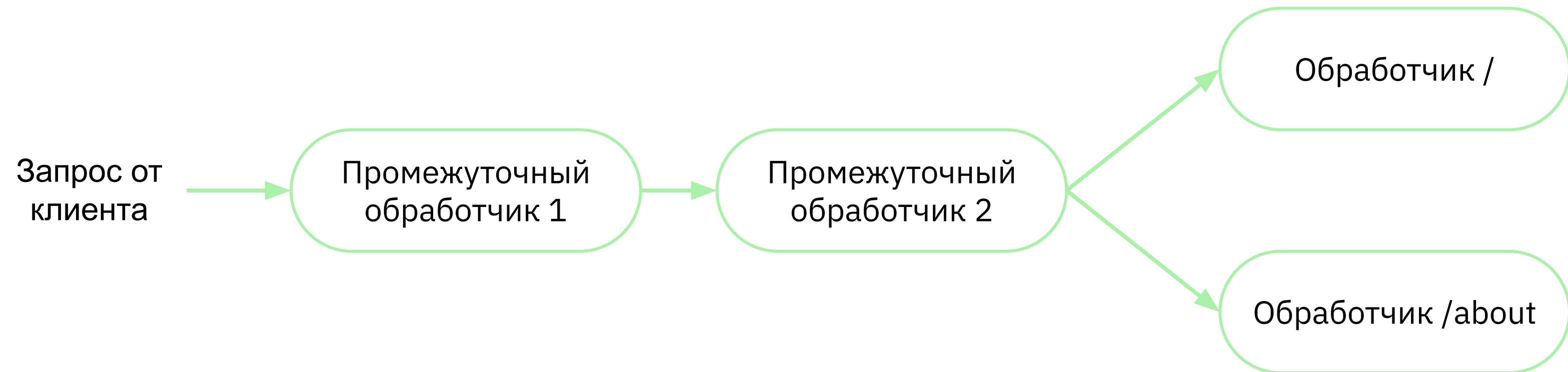
```
1 const express = require('express');
2
3 const app = express();
4
5 // Корневая страница
6 app.get('/', (req, res) => {
7     res.send('<h1>Добро пожаловать на мой сайт!</h1>');
8 });
9
10 // Страница "Обо мне"
11 app.get('/about', (req, res) => {
12     res.send('<h1>Страница обо мне!</h1>');
13 });
14
15 app.listen(port, () => {
16     console.log(`Сервер запущен на порту ${port}`);
17 });
```




Промежуточные обработчики

Express позволяет реализовать промежуточные обработчики запроса

Такие обработчики позволяют добавить некоторую логику перед основным обработчиком





Существует два вида промежуточных обработчиков

Глобальные промежуточные обработчики не привязаны к конкретному роуту

Локальные промежуточные обработчики определяются для конкретного роута



Переменные в URL

Глобальные промежуточные обработчики не привязаны к конкретному роуту

Локальные промежуточные обработчики определяются для конкретного роута



Возврат html файлов

Писать HTML в JavaScript строке не очень удобно.
Для удобства в express существует возможность отдавать в качестве ответа html файл с помощью метода `res.sendFile()`

```
1 const express = require('express');
2 const path = require('path');
3
4 const app = express();
5
6 app.get('/', (req, res) => {
7     res.sendFile(path.join(__dirname, 'static/index.html'));
8 });
9
10 const port = 3000;
11
12 app.listen(port, () => {
13     console.log(`Сервер запущен на порту ${port}`);
14 });
```



Определение

Статичные файлы - это файлы, которые хранятся на сервере и доступны для загрузки и просмотра из браузера





Статические файлы

Для того, чтобы определить директорию хранения статических файлов, необходимо использовать промежуточный обработчик **express.static()**

```
1 const express = require('express');
2 const path = require('path');
3
4 const app = express();
5
6 app.use(express.static('static'));
7
8 app.get('/', (req, res) => {
9     res.sendFile('static/index.html');
10 });
11
12 const port = 3000;
13
14 app.listen(port, () => {
15     console.log(`Сервер запущен на порту ${port}`);
16 });
```

Подведем итоги



Итоги

- Мы с вами разобрались с модулями и их видами
- Поработали с некоторыми встроенными модулями
- Узнали какие существуют глобальные объекты
- Установили express
- Разобрались с основными возможностями express



Что почитать на досуге?

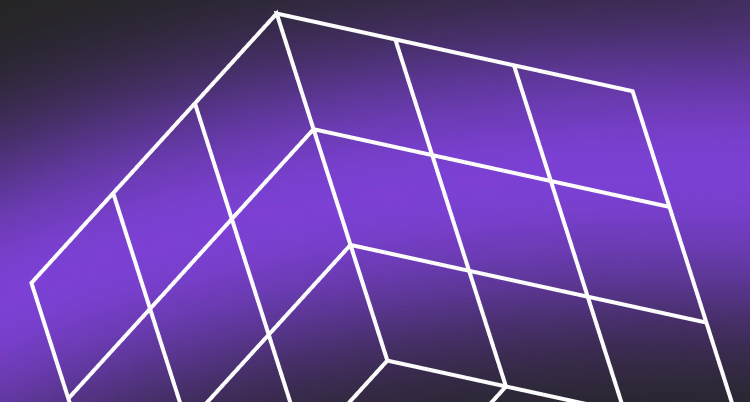
- Официальная документация о встроенных модулях Node.js <https://nodejs.org/dist/latest-v18.x/docs/api/>
- Официальная документация express <http://expressjs.com/ru/>





Домашнее задание

- Инициализируйте NPM проект и попробуйте установить пакет express
- Создайте файл index.js и импортируйте express
- Попробуйте написать http сервер на express с одним обработчиком





Спасибо за внимание

